DETAILED DESIGN

In a design document, a more detailed specification is given by explaining in natural language what a module is supposed to do. The first step before the detailed design or code for a module can be developed is that the specification of the module be given precisely. Once the module is precisely specified, the internal logic for the module that will implement the given specifications can be decided.

Detailed Design

Process Design Language (PDL) is one way in which the design can be communicated precisely and completely to whatever degree of detail desired by the designer. PDL is particularly useful when using top-down refinement techniques to design a system or module.

1. PDL

Ideally we would like to express the design in a language that is as precise and unambiguous as possible without having too much detail and that can be easily converted into an implementation. This is what PDL attempts to do.

PDL has an overall outer syntax of a structured programming language and has a vocabulary of a natural language (English in our case). As an example, consider the problem of finding the minimum and maximum of a set of numbers in a file and outputting these numbers in PDL as shown in Figure :

```
minmax(infile)
ARRAY a
  DO UNTIL end of input
     READ an item into a
  ENDDO
max, min :=first item of a
  DO FOR each item in a
      IF max<item THEN set max to item
      IF min>item THEN set min to item
  ENDDO
END
```

Figure : PDL description of the minmax program

PDL program we have the entire logic of the procedure, but little about the details of implementation in a particular language. To implement in a language, each of the PDL statements will have to be converted into programming language statements. The basic constructs of PDL are similar to those of a structured language. A variety of data structures can be defined and used in PDL such as lists, tables, scalar, and integers.

The basic constructs of PDL are similar to those of a structured language. The first is the IF construct. It is similar to the if-then-else construct of Pascal. However, the conditions and the

statements to be executed need not be stated in a formal language. For a general selection, there is a CASE statement. Some examples of CASE statements are:

CASE OF transaction type CASE OF operator type

The DO construct is used to indicate repetition. The construct is indicated by:

DO iteration criteria one or more statements ENDDO

The iteration criteria can be chosen to suit the problem, and unlike a formal programming language, they need not be formally stated. Examples of valid uses are:

DO WHILE there are characters in input file

DO UNTIL the end of file is reached

DO FOR each item in the list EXCEPT when item is zero

A variety of data structures can be defined and used in PDL such as lists, tables, scalar, and integers. Variations of PDL, along with some automated support, are used extensively for communicating designs.

2) Logic/Algorithm Design

The basic goal in detailed design is to specify the logic for the different modules that have been specified during system design. Specifying the logic will require developing an algorithm that will implement the given specifications.

An algorithm is a sequence of steps that need to be performed to solve a given problem. The problem need not be programming problem. A *procedure* is a finite sequence of well-defined steps or operations, each of which requires a finite amount of memory and time to complete.

There are number of steps that one has to perform while developing an algorithm.

1. ***Statement of the problem:*** The problem for which an algorithm is being devised has to be precisely and clearly stated and properly understood by the person responsible for designing the algorithm. For detailed design, the problem statement comes from the system design.
2. ***Development of a mathematical model:*** In modeling, one has to select the mathematical structures that are best suited for the problem.
3. ***Design of the algorithm:*** During this step the data structure and program structure are decided.

Once the algorithm is designed, its correctness should be verified. The most common method for designing algorithms or the logic for a module is to use the *stepwise refinement technique.*

The *stepwise refinement technique* breaks the logic design problem into a series of steps, so that the development can be done gradually. The *stepwise refinement technique* is a top-down method for developing detailed design.

- As an example let us consider the problem of counting different words in a text file.

- During detailed design we have to determine the logic of this module so that specifications are met. For this we use stepwise refinement technique.
- A simple strategy for the first step is shown

```
int count (file)
FILE file;
word_list wl;
{
   read file into wl
   sort (wl);
   count = different_words (wl);
   printf (count);
}
```

FIGURE 7.5. Strategy for the first step in stepwise refinement.

Figure (a)

There are 3 steps that need refinement. They are

1. Read file into a word list (whose purpose is to read all the words from the file and create a word list).
2. Sort(wl) which sorts in ascending order
3. Count different words from the sorted list. So far,only one data structure is defined.

```
read_from_file (file, wl)
FILE file;
word_list wl;
{
   initialize wl to empty;
   while not end-of-file {
   get_a_word from file
   add word to wl
}
```

FIGURE 7.6. Refinement of the reading operation.

Figure (b)

In the next refinement step,we should select one of the three operations to be refined-and future elaboratesit. The module is the read module is to read words and add them to the word list.

```
int different_words (wl)
word_list wl;
{
  word last, cur;
  int cnt;

  last = first word in wl
  cnt = 1;
  while not end of list {
    cur = next word from wl
    if (cur <> last) {
      cnt = cnt + 1;
      last = cur;
    }
  }
  return (cnt)
}
```

FIGURE 7.7. Refinement of the function different_words

Figure (c) : Here we refine the sort function once the refinement are done,we have a design that is sufficiently detailed and needs no further refinements.

3) State Modeling of Classes:

The technique for getting a more detailed understanding of the class as a whole, without talking about the logic of different methods, has to be different from the refinement-based approach. An object of a class has some state and many operations on it.

A method to understand the behavior of a class is to view it as finite state automata (FSA). An FSA consists of states and transitions between states, which take place when some events occur. When modehng an object, the state is the value of its attributes, and an event is the performing of an operation on the object. A state diagram relates events and states by showing how the state changes when an event is performed. A state diagram for an object will generally have an initial state, from which all states in the FSA are reachable.

For example, for an object that represents a stack, all states that represent a stack of size more than 0 and less than some defined maximum are similar as the behavior of all operations defined on the stack will be similar in all such states.

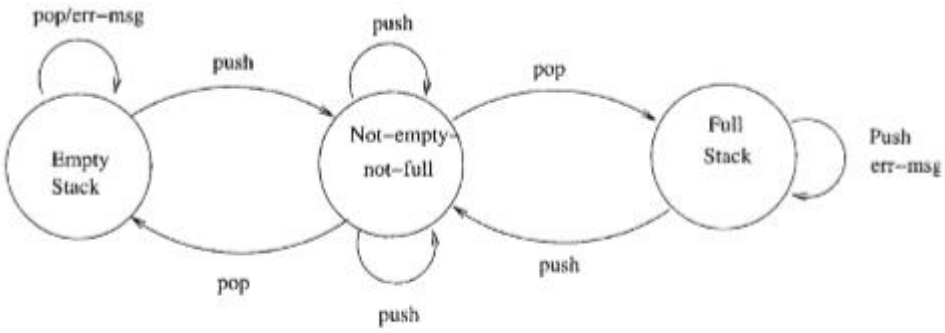


Figure 8.6: FSA model of a stack.

Verification

There are few techniques available to verify that the detailed design is consistent with the system design. The focus of verification in the detailed design phase is on showing that the detailed design meets the specifications laid down in the system design. The three verification methods we consider are design walkthroughs, critical design review, and consistency checkers.

The 3 verification method considered are:

1) ***Design Walkthroughs***

2) ***Critical Design Review***

3) ***Consistency Checkers***

Design Walkthroughs

- A manual method of verification.
- The definition and use of walkthroughs change from organization to organization.
- A design walkthrough is done in an informal meeting called by the designer or the leader of the designer's group.

❖ In a walkthrough
  - the designer explains the logic step by step, and
  - the members of the group ask questions,
  - point out possible errors or seek clarification.

❖ A beneficial side effect of walkthroughs is -
  - in the process of articulating and explaining the design in detail,
  - the designer himself can uncover some of the errors.

Critical Design Review

The purpose of critical design review is to ensure that the detailed design satisfies the specifications laid down during system design. The critical design review process is same as the inspections process in which a group of people get together to discuss the design with the aim of revealing design errors or undesirable properties.

The review group includes, besides the author of the detailed design, a member of the system design team, the programmer responsible for ultimately coding the module(s) under review, and an independent software quality engineer.

The use of checklists, as with other reviews, is considered important for the success of the review. The checklist is a means of focusing the discussion or the "search" of errors.Checklists can be used by each member during private study of the design and during the review meeting. For best results, the checklist should be tailored to the project at hand, to uncover projectspecific errors.

A Sample Checklist

• Does each of the modules in the system design exist in detailed design?
• Are there analyses to demonstrate that the performance requirements can be met?
• Are all the assumptions explicitly stated, and are they acceptable?
• Are all relevant aspects of system design reflected in detailed design?
• Have the exceptional conditions been handled?
• Are all the data formats consistent with the system design?
• Is the design structured, and does it conform to local standards?
• Are the sizes of data structures estimated? Are provisions made to guard against overflow?
• Is each statement specified in natural language easily codable?

• Are the loop termination conditions properly specified?
• Are the conditions in the loops OK?
• Are the conditions in the if statements correct?
• Is the nesting proper?
• Is the module logic too complex?
• Are the modules highly cohesive?

Consistency Checkers

- Consistency checkers are essentially compilers that take as input the design specified in a design language (PDL in our case).
- They cannot produce executable code because the inner syntax of PDL allows natural language and many activities are specified in the natural language.
- A consistency checker can ensure that any modules invoked or used by a given module actually exist in the design and that the interface used by the caller is consistent with the interface definition of the called module.
- Depending on the precision and syntax of the design language, consistency checkers can produce other information.
- In addition, these tools can be used to compute the complexity of modules and other metrics, because these metrics are based on alternate and loop constructs, which have a formal syntax in PDL.

****

Chapter 2 CODING

The goal of the coding or programming phase is to translate the design of the system produced during the design phase into code in a given programming language, which can be executed by a computer and that performs the computation specified by the design.

- The goal of the coding or programming activity is to implement the design in the best possible manner. The coding activity affects both testing and maintenance profoundly. As we saw earlier, the time spent in coding is a small percentage of the total software cost, while testing and maintenance consume the major portion. Thus, it should be clear that the goal during coding shouldn't be just to reduce the implementation cost, but help reduce the cost of later phases (reduce the work of tester and maintainer).

During implementation, it should be kept in mind that *the programs should not be constructed so that they are easy to write, but so that they are easy to read and understand.* There are many different criteria for judging a program, including readability, size of the program, execution time, and required memory.

Programming Principles and Guidelines

The primary goal of the coding phase is to translate the given design into source code in a given programming language, so that code is simple, easy to test, and easy to understand and modify.

Good programming is a skill that can only be acquired by practice. However, much can be learned from the experience of others, and some general rules and guidelines can be laid for the programmer.

Good programming(producing correct and simple programs) is a practice independent of the target programming language, although well-structured programming languages make the programmer's job simpler.

a) Common Coding Errors

Software errors are a reality that all programmers have to deal with. Much of effort in developing software goes in identifying and removing bugs.

Memory Leaks

A memory leak is a situation where the memory is allocated to the program which is not freed subsequently. This error is a common source of software failures which occurs frequently in the languages which do not have automatic garbage collection (like C, C++).

A software system with memory leaks keeps consuming memory, till at some point of time the program may come to an exceptional halt because of the lack of free memory. An example of this error is:

```
char* foo(int s)
{
        char *output;
        if (s>0)
          output=(char*) malloc (size);
        if (s==1)
          return NULL; /* if s==1 then mem leaked */
        return(output);
}
```

Freeing an Already Freed Resource

In general, in programs, resources are first allocated and then freed. For example, memory is first allocated and then deallocated. This error occurs when the programmer tries to free the already freed resource.

```
main()
{
  char *str;
  str=(char *)malloc(10);
  if (global==0)
        free(str);
  free(str); /* str is already freed
}
```

NULL Dereferencing

This error occurs when we try to access the contents of a location that points to NULL. This is a commonly occurring error which can bring a software system down. It is also difficult to detect as it the NULL dereferencing may occur only in some paths and only under certain situations.It can also be caused because of aliases—for example, two variables refer to the same object, and one is freed and an attempt is made to dereference the second. This code segment shows two instances of NULL dereference.

```
char *ch=NULL;
if (x>0)
{
  ch='c';
}
printf("\%c", *ch); /* ch may be NULL
*ch=malloc(size);
ch = 'c'; /* ch will be NULL if malloc returns NULL
```

This often occurs if data is initialized in most cases, but some cases do not get covered, they were not expected. An example of this error is:

```
switch( i )
{
       case 0: s=OBJECT_1; break;
       case 1: s=OBJECT_2;break;
}
return (s); /* s not initialized for values
               other than 0 or 1 */
```

Lack of Unique Addresses

Aliasing creates many problems, and among them is violation of unique addresses when we expect different addresses. For example in the string concatenation function, we expect source and destination addresses to be different.

strcat (src ,destn) ; /* In above function, if src is aliased to destn,
* then we may get a runtime error */

Synchronization Errors

In a parallel program, where there are multiple threads possibly accessing some common resources, then synchronization errors are possible.There are different categories of synchronization errors, some of which are:

1. Deadlocks
2. Race conditions
3. Inconsistent synchronization

Deadlock is a situation in which one or more threads mutually lock each other. The most frequent reason for the cause of deadlocks is inconsistentlocking sequence—the threads in deadlock wait for resources which are in turn locked by some other thread.

Race conditions occur when two threads try to access the same resource and the result of the execution depends on the order of the execution of the threads.

Inconsistent synchronization is also a common error representing the situation where there is a mix of locked and unlocked accesses to some shared variables, particularly if the access involves updates.

Array Index Out of Bounds

Array index often goes out of bounds, leading to exceptions. Care needs to be taken to see that the array index values are not negative and do not exceed their bounds.

Arithmetic exceptions

These include errors like divide by zero and floating point exceptions. The result of these may vary from getting unexpected results to termination of the program.

Off by One

This is one of the most common errors which can be caused in many ways. For example, starting at 1 when we should start at 0 or vice versa, writing <— N instead of < N or vice versa, and so on.

Enumerated data types

Overflow and underflow errors can easily occur when working with enumerated types, and care should be taken when assuming the values of enumerated data types. An example of such an error is:

```
typedef enum {A, B,C, D} grade;
void foo(grade x)
{
      int l,m;
      l=GLOBAL_ARRAY[x-1]; /* Underflow possible */
      m=GLOBAL_ARRAY[x+1]; /* Overflow possible */
}
```

Illegal use of &; instead of &&:

This bug arises if non short circuit logic (like & or |) is used instead of short circuit logic (&& or ||). Non short circuit logic will evaluate both sides of the expression. But short circuit operator evaluates one side and based on the result, it decides if it has to evaluate the other side or not.

if(object!= null &object.getTitle() != null)
/* Here second operation can cause a null dereference */

String handling errors

There are a number of ways in which string handling functions like strcpy, sprintf, gets etc can fail. Examples are one of the operands is NULL, the string is not NULL terminated, or the source operand may have greater size than the destination.

Buffer overflow

When a program takes an input which is being copied in a buffer, by giving a large (and malicious) input, a malicious user can overflow the buffer on the stack. By doing this, the return address can get rewritten to whatever the malicious user has planned. So, when the function call ends, the control goes to where the malicious user has planned, which is typically some malicious code to take control of the computer or do some harmful actions.

```
void mygets(char *str){
      int ch;
      while(ch=getchar() !='\n' && ch!='\0')
            *(str++)=ch;
      *str='\0';
}
main(){
      char s2[4];
      mygets(s2);


}
```

Here there is a possible buffer overflow attack. If the input given is large, it can overflow the buffer s2, and by carefully crafting the bytes that go on the stack the return address of mygets() can be replaced by an address of a malicious program.

b) ***Structured Programming***

A program has a static structure as well as a dynamic structure. The static structure is the structure of the text of the program, which is usually just a linear organization of statements of the

program. The dynamic structure of the program is the sequence of statements executed during the execution of the program.

The goal of structured programming is to ensure that the static structure and the dynamic structures are the same. That is, the objective of structured programming is to write programs so that the sequence of statements executed during the execution of a program is the same as the sequence of statements in the text of that program.

The key property of a structured statement is that it has a *single-entry* and a *single-exit.* The most commonly used single-entry and single-exit statements are:

Selection: if B then S1 else S2
If B then S1

Iteration: While B do S
Repeat S until B

Sequencing: S1; S2; S3; …

The basic goal, as we have tried to emphasize, is to make the logic of the program simple to understand. No hard-and-fast rule can formulated that will be applicable under all circumstances.

The basic objective of using structured constructs is to linearize the control flow so that the execution behavior is easier to understand and argue about. There are some common programming practices that are now well understood that make use of unstructured constructs (e.g., break and continue statement). Although efforts should be made to avoid using statements that effectively violate the single-entry single-exit property, if the use of such statements is the simplest way to organize the program, then from the point of view of readability, the constructs should be used.

c) ***Information Hiding***

Any software solution to a problem contains data structures that represent information in the problem domain. Any information in the problem domain typically has a small number of defined operations performed on it.

When the information is represented as data structures, the principle should be applied, and only some defined operations should be performed on the data structures. This is the principle of information hiding. The information captured in the data structures should be hidden from the rest of the system, and only the access functions on the data structures that represent the operations performed on the information should be visible.

If the information hiding principle is used, the data structure need not be directly used and manipulated by other modules. All modules, other than the access functions, access the data structure through the access functions.

Information hiding can reduce the coupling between modules and make the system more maintainable.When a data structure is changed, the effect of the change is generally limited to the access functions if information hiding is used. Otherwise, all modules using the data structure may have to be changed.

Information hiding is also an effective tool for managing the complexity of developing software.

Another form of information hiding is to let a module see only those data items needed by it. The other data items should be "hidden" from such modules and the modules should not be allowed to access these data items. Thus, each module is given access to data items on a "need-to-know" basis.

With support of data abstraction, a package or a module is defined that encapsulates the data. Some operations are defined by the module on the encapsulated data. Other modules that are outside this module can only invoke these predefined operations on the encapsulated data. The advantage of this form of data abstraction is that the data is entirely in the control of the module in which the data is encapsulated. Other modules cannot access or modify the data; the operations that can access and modify are also a part of this module.

d)Some Programming Practices (OR) (RULES FOR WRITING CODE)

It is impossible to provide an exhaustive list of what to do and what not to do to produce simple readable code. Some general rules that usually apply:

Names: Selecting module and variable names is often not considered important by novice programmers. Variable names should be closely related to the entity they represent, and module names should reflect their activity.

Control Constructs: It is desirable that as much as possible single-entry, single-exit constructs be used. It is also desirable to use a few standard control constructs.

Gotos: Gotos should be used sparingly and in a disciplined manner. If a goto must be used, a forward transfer (or a jump to a later statement) is more acceptable than a backward jump. Use of gotos for exiting a loop or for invoking error handlers is quite acceptable.

Information Hiding: Information hiding should be supported where possible. Only the access functions for the data structures should be made visible while hiding the data structure behind these functions.

User-defined Types: Modern languages allow users to define types like the enumerated type.**E.G : TYPE DAYS=(MON,TUE,WED,THURS..);**

Nesting: The different control constructs, particularly the if-then-else, can be nested. If the nesting becomes too deep, the programs become harder to understand. If possible, deep nesting should be avoided.

If C1 then s1

Elseif c2 then s2

Else if c3 then s3

Else if c4 then s4;

Module Size: A programmer should carefully examine any routine with very few statements or with too many statements. Large modules often will not be functionally cohesive, and too-small modules might incur unnecessary overhead.

Module Interface: A module with a complex interface should be carefully examined. Such modules might not be functionally cohesive and might be implementing multiple functions.

Program Layout: How the program is organized and presented can have great effect on the readability of it. Proper identification, blank spaces, and parentheses should be used to enhance the readability of programs.

Side Effects: When a module is invoked, it sometimes has side effects of modifying the program state beyond the modification of parameters listed in the module interface definition, for example, modifying global variables.

Robustness: A program is robust if it does something planned even for exceptional conditions. A program should check for validity of inputs, where possible, and should check for possible overflow of the data structures. If such situations do arise, the program should not just “crash” or “core dump”; it should produce some meaningful message and exit gracefully.

Switch case with default: If there is no default case in a "switch" statement, the behavior can be unpredictable if that case arises at some point of time which was not predictable at development stage. Such a practice can result in a bug like NULL dereference, memory leak, as well as other types of serious bugs. It is a good practice to always include a default case.

```
switch (i){
      case 0 : {s=malloc(size)
      }
 s[0] = y; /* NULL dereference if default occurs*/
```

Empty Catch Block: An exception is caught, but if there is no action, it may represent a scenario where some of the operations to be done are not performed. Whenever exceptions are caught, it is a good practice to take some default action, even if it is just printing an error message.

```
try {
      FileInputStream fis = new
      FileInputStream("InputFile");
}
catch (IOException ioe) { }
      // not a good practice
```

Empty if, while Statement: A condition is checked but nothing is done based on the check. This often occurs due to some mistake and should be caught. Other similar errors include empty finally, try, synchronized, empty static method, etc. Such useless checks should be avoided.

```
if (x == 0) {}  /* nothing is done after checking x */
else {
   :
}
```

Read Return to be Checked: Often the return value from reads is not checked, assuming that the read returns the desired values. Sometimes the result from a read can be different from what is expected, and this can cause failures later. There may be some cases where neglecting this condition may result in some serious error. For example, if read from scanf() is more than expected, then it may cause a buffer overflow. Hence the value of read should be checked before accessing the data read.

Return From Finally Block: One should not return from finally block, as cases it can create false beliefs. For example, consider the code

```
public String foo() {
      try {
              throw new Exception( "An Exception" );
      }
      catch (Exception e) {
              throw e;
      }
      finally {
              return "Some value";
              }
      }
```

In this example, a value is returned both in exception and non exception scenarios. When we have a return from try block, in this case, if there is a return in finally also, then the value from finally is returned instead of the value from try.

Correlated Parameters: Often there is an implicit correlation between the parameters. For example, in the code segment given below, "length" represents the size of BUFFER. If the correlation does not hold, we can run into a serious problem like buffer overflow. In general, it is desirable to do some counter checks on implicit assumptions about parameters.

```
void (char *src, int length, char destn[]) {
      strcpy (destn, src); /* Can cause buffer overflow
                           if length > MAX_SIZE */
}
```

Trusted Data sources: Counter checks should be made before accessing the input data, particularly if the input data is being provided by the user or is being obtained over the network. For example, while doing the string copy operation, we should check that the source string is null terminated, or that its size is as we expect.

Give Importance to Exceptions: Most programmers tend to give less attention to the possible exceptional cases and tend to work with the main flow of events, control, and data. Though the main work is done in the main path, it is the exceptional paths that often cause software systems to fail.

Give Importance to Exceptions: Most programmers tend to give less attention to the possible exceptional cases and tend to work with the main flow of events, control, and data. Though the main work is done in the main path, it is the exceptional paths that often cause software systems to fail.

Coding Standards

Programmers spend far more time reading code than writing code. Over the life of the code, the author spends a considerable time reading it during debugging and enhancement. It is of prime importance to write code in a manner that it is easy to read and understand. Coding standards provide rules and guidelines for some aspects of programming in order to make code easier to read.

In general, coding standards provide guidelines for programmers regarding naming, file organization, statements and declarations, and layout and comments.

Naming Conventions

Some of the standard naming conventions that are followed often are:

- Package names should be in lower case (e.g., mypackage, edu.iitk.maths)
- Type names should be nouns and should start with uppercase (e.g., Day, DateOfBirth, EventHandler)
- Variable names should be nouns starting with lower case (e.g., name, amount)
- Constant names should be all uppercase (e.g., PI, MAXJTERATIONS)
- Method names should be verbs starting with lowercase (e.g., getValue())
- Private class variables should have the _ suffix (e.g., "private int value_"). (Some standards will require this to be a prefix.)

- Variables with a large scope should have long names; variables with a small scope can have short names; loop iterators should be named i, j, k, etc.
- The prefix is should be used for boolean variables and methods to avoid confusion (e.g., isStatus should be used instead of status); negative boolean variable names (e.g., isNotCorrect) should be avoided.
- The term compute can be used for methods where something is being computed; the term find can be used where something is being looked up (e.g., computeMean(), findMin().)
- Exception classes should be suffixed with Exception (e.g., OutOfBoundException.)

Files

There are conventions on how files should be named, and what files should contain, such that a reader can get some idea about what the file contains. Some examples of these conventions are:

• Java source files should have the extension .Java—this is enforced by most compilers and tools.

• Each file should contain one outer class and the class name should be same as the file name.

• Line length should be limited to less than 80 columns and special characters should be avoided. If the line is longer, it should be continued and the continuation should be made very clear.

Statements

These guidelines are for the declaration and executable statements in the source code.

Variables should be initialized where declared, and they should be declared in the smallest possible scope.

• Declare related variables together in a common statement. Unrelated variables should not be declared in the same statement.

• Class variables should never be declared pubhc.

• Use only loop control statements in a for loop.

• Loop variables should be initialized immediately before the loop.

• Avoid the use of break and continue in a loop.

• Avoid the use of do ... while construct.

• Avoid complex conditional expressions—introduce temporary boolean variables instead.

• Avoid executable statements in conditionals.

Commenting and Layout

Comments are textual statements that are meant for the program reader to aid the understanding of code. In general, comments should explain what the code is doing or why the code is there, so that the code can become almost standalone for understanding the system. Comments should generally be provided for blocks of code, and in many cases, only comments for the modules need to be provided.

Providing comments for modules is most useful, as modules form the unit of testing, compiling, verification and modification. Comments for a module are often called prologue for the module, which describes the functionality and the purpose of the module, its public interface and how the module is to be used, parameters of the interface, assumptions it makes about the parameters, and any side effects it has.

Java provides documentation comments that are delimited by "/** ... */", and which could be extracted to HTML files.

In addition to prologue for modules, coding standards may specify how and where comments should be located. Some such guidelines are:

• Single home comments for a block of code should be aligned with the code they are meant for.

• There should be comments for all major variables explaining what they represent.

• A block of comments should be preceded by a blank comment line with just "/*" and ended with a line containing just "*/".

• Trailing comments after statements should be short, on the same line, and shifted far enough to separate them from statements.

Coding Process

The coding activity starts when some form of design has been done and the specifications of the modules to be developed are available. With the design, modules are usually assigned to individual developers for coding. In a top-down implementation, we start by assigning modules at the top of the hierarchy and proceed to the lower levels. In a bottom-up implementation, the development starts with first implementing the modules at the bottom of the hierarchy and proceeds up.

1. **An Incremental Coding Process**

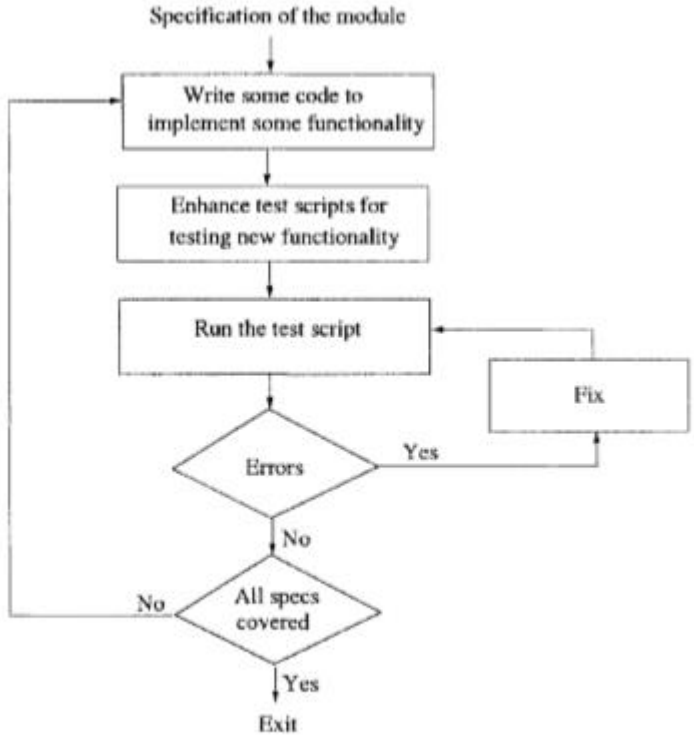


Figure 9.2: An incremental coding process.

The process followed by many developers is to write the code for the currently assigned module, and when done, perform unit testing on it and fix the bugs found. Then the code is checked in the project repository to make it available to others in the project.

A better process for coding that is often followed by experienced developers is to develop the code incrementally. That is, write code for implementing only part of the functionality of the module. This code is compiled and tested with some quick tests to check the code that has been written so far. When the code passes these tests, the developer proceeds to add further functionality to the code, which is then tested again. In other words, the code is built incrementally by the developers, testing it as it is built.

The basic advantage of developing code incrementally with testing being done after every round of coding is to facilitate debugging—an error foundin some testing can be safely attributed to code that was added since last successful testing.

For following this process, it is essential that there be automated test scripts that can run the test cases with the click of a button. With these test scripts, testing can be done as frequently as desired and new test cases can be added easily.

2. Test Driven Development

Test Driven Development (TDD) [11] is a coding process that turns around the common approach to coding. In TDD, a programmer first writes the test scripts, and then writes the code to pass the tests. The whole process is done incrementally, with tests being written based on the specifications and code being written to pass the tests.



Figure 9.3: Test driven development process.

First, the approach says that we write just enough code to pass the tests. By following this, the code is always in sync with the tests. This is not always the case with the code-first approach, in which it is all too common to write a long piece of code, but then only write a few tests which cover only some parts of the code.The task of test cases to check that the code that will be developed has all the functionality needed.

This writing of test cases before the code is written makes the development usage-driven. That is, first the focus is to determine how the code to be developed will be used. This is extracted from the specifications and the usage interface is specified precisely when the test cases are written.The focus is on the users of the code and the code is written to satisfy the users. This can reduce interface errors.

In TDD, the first few tests are likely to focus on using the main functionality. Generally, the test cases for lower priority features or functionality will be developed later.Consequently, code for high priority features will be developed first and lower priority items will be developed later.

As the code is written to satisfy the test cases, the completeness of the code depends on the thoroughness of the test cases. Often it is hard and tedious to write test cases for all the scenarios or

special conditions, and it is highly unlikely that a developer will write test cases for all the special cases.

In TDD, as the goal is to write enough code to pass the test cases, such special cases may not get handled. Also, as at each step code is being written primarily to pass the tests, it may later be found that earlier algorithms were not well suited.

3. Pair Programming

- Pair programming is also a coding process that has been proposed as a key technique in extreme programming (XP) methodology.
- In pair programming, code is not written by individual programmers but by a pair of programmers.
- The process envisaged is that one person will type the program while the other will actively participate and constantly review what is being typed.
- When errors are noticed, they are pointed out and corrected. When needed, the pair discusses the algorithms, data structures, or strategies to be used in the code to be written.
- The roles are rotated frequently making both equal partners and having similar roles.
- The basic motivation for pair programming is that as code reading and code reviews have been found to be very effective in detecting defects, by having a pair do the programming we have the situation where the code is getting reviewed as it is being typed.
- Besides ongoing code review, having two programmers apply themselves to the programming task at hand is likely to result in better decisions being taken about the data structures, algorithms, interfaces, logic, etc.
- The potential drawback of pair programming is that it may result in loss of productivity by assigning two people for a programming task.
- It is clear that a pair will produce better code as compared to code being developed by a single programmer.
- There are also issues of accountability and code ownership, particularly when the pairs are not fixed and rotate.
- Impact of pair programming is an active area of research, particularly for experimental software engineering.

Source Code Control and Build

- In a project many different people develop source code. Each programmer creates different source files, which are eventually combined together to create executables.
- In order to keep control over the sources and their evolution, source code control is almost always used in projects using tools like the CVS on UNIX (www.cvshome.org) or visual source safe (VSS) on Windows (msdn.microsoft.com/vstudio/previous/ssafe).
- A modern source code control system contains a repository, which is essentially a controlled directory structure, which keeps the full revision history of all the files.
- For a project, a repository has to be set up with permissions for different people in the project. The files the repository will contain are also specified— these are the files whose evolution the repository maintains.

Some of the types of commands that are generally performed by a programmer are:

- **Get a local copy:** A programmer in a project works on a local copy of the file. Commands are provided to make a local copy from the repository. Making a local copy is generally called a checkout.
  An example command is cvs checkout < module >, which copies a set of files that belongs to the < module > on the local machine.
  A user will get the latest copy of the file. However, if a user wants, any older version of a file can be obtained from the repository, as the complete history is maintained.
- **Make changes to file(s).** The changes made to the local file by a programmer remain local until the changes are committed back on the repository.
  By committing (e.g., by cvs commit < file >) the changes made to the local file are made to the repository, and are hence available to others. This operation is also referred to as check in.
- **Update a local copy.** Changes committed by project members to the repository are not reflected in the local copies that were made before the changes were committed. For getting the changes, the local copies of the files have to be updated (e.g., by cvs update command).
- **Get Reports.** Source control tools provide a host of commands to provide different reports on the evolution of the files. These include reports like the difference between the local file and the latest version of the file, all changes made to a file along with the dates and reasons for change.

Once the changes are committed, they become available to all members of the team, who are generally supposed to use the source files from the repository for checking their own programs.

In steady state, the normal behavior of a project member will be as follows: check out the latest version of the files to be changed; make the planned changes to them; validate that the changes have the desired effect (for which all the files may be copied and the system tried out locally); commit the changes back to the repository.

It should be clear that if two people check out some file and then make changes, there is a possibility of a conflict—different changes are made to the same parts of the file. All tools will detect the conflict when the second person tries to commit the changes, and will inform the user.The user has to manually resolve the conflit, i.e., make the file such that the changes do not conflict with existing changes, and then commit the file.

With a source code control system, a programmer does not need to maintain ah the versions—at any time if some changes need to be undone, older versions can be easily recovered.

Besides using the repository for maintaining the different versions, it is also used for constructing the software system from the sources—an activity often called build. Building the final executables from the source files is often done through tools like the Makefile, which specify the dependence between files and how the final executables are constructed from the source files. With source code control, these tools will generally get the latest copy from the repository, and then use it for creating executables.

Refactoring

Refactoring is the technique to improve existing code and prevent this design decay with time. Refactoring is part of coding in that it is performed during the coding activity, but is not

regular coding. Refactoring has been practiced in the past by programmers, but recently it has taken a more concrete shape, and has been proposed as a key step in the Extreme Programming practice.

Basic Concepts

Refactoring is defined as a change made to the internal structure of software to make it easier to understand and cheaper to modify without changing its observable behavior. The basic objective of refactoring is to improve the design. However, note that this is not about improving a design during the design stages for creating a design which is to be later implemented but about improving the design of code that already exists. A refactoring generally results in one or more of the following:

1. Reduced coupling
2. Increased cohesion
3. Better adherence to open-closed principle (for 00 systems)

Refactoring involves changing the code to improve one of the design properties, while keeping the external behavior the same. Even though refactoring is triggered by the need to change the software (and its external behavior), it should not be confused or mixed with the changes for enhancements. It is best to keep these two types of changes separate. So, while developing code, if refactoring is needed, the programmer should cease to write new functionality, and first do the refactoring, and then add new code.

The main risk of refactoring is that existing working code may "break" due to the changes being made. To mitigate this risk, the two golden rules are:

1. Refactor in small steps
2. Have test scripts available to test existing functionality

If a good test suite is available, then whether refactoring preserves existing functionality can be checked easily. Refactoring cannot be done effectively without an automated test suite as without such a suite determining if the external behavior has changed or not will become a costly affair.

Often the designers spend considerable effort in trying to make the design as good as possible, try to think of future changes and try to make the design flexible enough to accommodate all types of future changes they can envisage. This makes the design activity very complex, and often results in complex designs.

Bad Smells

1. **Duplicate Code.** This is quite common. One reason for this is that some small functionality is being executed at multiple places. Another common reason is that when there are multiple subclasses of a class, then each subclass may end up doing a similar thing. Duplicate code means that if this logic or function has to be changed, it will have to be changed in all the places it exists, making changes much harder and costlier.
2. **Long Method.** If a method is large, it often represents the situation where it is trying to do too many things and therefore is not cohesive.
3. **Long Class.** Similarly, a large class may indicate that it is encapsulating multiple concepts, making the class not cohesive.
4. **Long Parameter List.** Complex interfaces are clearly not desirable— they make the code harder to understand. Often, the complexity is not intrinsic but a sign of improper design.
5. **Switch Statements.** In object-oriented programs, if the polymorphism is not being used properly, it is likely to result in a switch statement everywhere the behavior is to be different depending on the property.

6. **Speculative Generality.** Some class hierarchies may exist because the objects in subclasses seem to be different. However, if the behavior of objects of the different subclasses is the same, and there is no immediate reason to think that behaviors might change, then it is a case of unnecessary complexity.
7. **Too Much Communication Between Objects**. If methods in one class are making many calls to methods of another object to find out about its state, this is a sign of strong coupling. It is possible that this may be unnecessary and hence such situations should be examined for refactoring.
8. **Message Chaining.** One method calls another method, which simply passes this call to another object, and so on. This chain potentially results in unnecessary coupling.
   These bad smells in general are indicative of a poor design.

Common Refactoring

A refactoring should help make the code easier to understand and modify. To achieve this objective, many refactorings focus on improving the methods, classes, or class hierarchies.

Improving Methods:

1. Extract Method. This refactoring is often done if a method is too long, indicating that it may not be cohesive and may be performing multiple functions. During this refactoring, a piece of code from a method is extracted out as a new method. The variables referred to in this code that are in the scope of original method become the parameters to the new method.
   In the original method, the extracted code is replaced by an invocation to the new method. Sometimes, the new method may be a function returning a value. If there is a method that returns a value but also changes the state of some objects, then this method should be converted into two methods—one that returns the desired value, and the other to make the desired state change
2. **Add/Remove Parameter.** If a method needs more information from its caller, perhaps because the scope of what the method has to do has expanded, then new parameters need to be added. However, this should be done only if the existing parameters cannot provide the information that is needed. The dual of this is parameter removal.
   For the sake of simplicity, parameters that are not being used should be removed. But if a class is part of a hierarchy, this has to be done carefully to make sure that hierarchy relationships are not disturbed

Improving Classes

Enhancing the cohesion of classes often results in moving fields and methods from one class to another such that logically connected data items and methods that access them are all encapsulated together in one object.

1. **Move Method.** In this a method is moved from one class to another. This is a very important refactoring, and is generally done when a method in class A seems to be interacting too heavily with another object of class B, indicating that perhaps the natural

home for the method is class B and not class A. Initially, it is better to leave the source method as a delegating method which just invokes the new method. This way, the change is limited only to the two methods.

2. **Move Field.** If a field in class A is being used more often by methods in class B, then the field should be moved to class B. This will reduce the coupling between A and B, and enhance the encapsulation of both the classes. This refactoring is similar to the one above—in Move Method the behavior is being reassigned and in this one state is being reassigned.
3. **Extract Class.** Often a designer starts with a class and as the need for new features arise, the classes are extended to do more, sometimes resulting in large classes that do not have clear abstraction and are holding too many responsibilities. If there is a large class that seems to be holding multiple responsibilities or encapsulating more than one concept, then this refactoring is applied. A new class is created and the relevant fields and methods are moved to the new class. One way to perform this refactoring is to first create the new class and create a link from the old class to the new one. Then the move field and move method refactorings can be repeatedly applied to move the fields and methods that belong to the new class.
4. **Replace Data Value with Object**. This is similar to the Extract Class refactoring. Often some data items are treated as fields of the class initially. As development proceeds, these data items become semantically richer with more operations being performed on them. If multiple operations are being performed within the class on these data items, then it may make sense to convert the data into an object.

Improve Hierarchies

Class hierarchy is a key object oriented concept that is the foundation for the open-closed principle. In order to support this principle, it is imperative that polymorphism be used effectively.

1. **Replace Conditional with Polymorphism**. If we have a class in which the behavior of some methods depends on value of some type-code, we essentially have a situation where a traditional, function-oriented approach is still being used. Polymorphism can, and should, be used to capture the situation more naturally.
2. **Pull up Field/Method.** An important factor in having a good hierarchy is to have common elements belonging to parent class while the variable elements belonging to the sub classes. when there is a situation that multiple subclasses have the same field, the field should be pulled up to the super class. Similarly, if there are methods in subclasses that are producing identical results we have a situation where functionality is being duplicated. And duplicate code is one of the key factors that makes making changes much harder.

   Hence, such a situation exists, the structure is improved if the method is moved up to the super class. If the subclasses have constructors which are similar, then they can also be pulled up into the super class and be called from the subclass constructor

Verification

Verification of the output of the coding phase is primarily intended for detecting errors introduced during this phase. The goal of verification of the code produced is to show that the code is consistent with the design it is supposed to implement.

Code Inspections:

Code inspections are usually held after code has been successfully compiled and other forms of static tools have been applied. The main motivation for this is to save human time and effort, which would otherwise be spent detecting errors that a compiler or static analyzer can detect.

The documentation to be distributed to the inspection team members includes the code to be reviewed and the design document. The team for code inspection should include the programmer, the designer, and the tester. The aim of code inspections is to detect defects in code. Often the type of defects the code inspection should focus on is contained in a checklist that is provided to the inspectors. Some of the items that can be included in a checklist for code reviews are:

A Sample Checklist:

- Do data definitions exploit the typing capabilities of the language?
- Do all the pointers point to some object? (Are there any "dangling pointers"?)
- Are the pointers set to NULL where needed?
- Are pointers being checked for NULL when being used?
- Are all the array indexes within bound?
- Are indexes properly initialized?
- Are all the branch conditions correct (not too weak, not too strong)?
- Will a loop always terminate (no infinite loops)?
- Is the loop termination condition correct?
- Is the number of loop executions "off by one" ?
- Where applicable, are the divisors tested for zero?
- Are imported data tested for validity?
- Do actual and formal interface parameters match?
- Are all variables used? Are all output variables assigned?
- Can statements placed in the loop be placed outside the loop?
- Are the labels unreferenced?
- Will the requirements of execution time be met?
- Are the local coding standards met?

Program verification methods fall into two categories:

1. **Static methods**
2. **Dynamic methods**

In dynamic methods the program is executed on some data and the outputs of the program are examined to determine if there are any errors present. Hence, dynamic techniques follow the traditional pattern of testing.

Static techniques do not involve actual program execution on actual numeric data, though it may involve some form of conceptual execution. Common forms of static techniques are program verification, code reading, code reviews and walkthroughs, and symbolic execution.

In static techniques often the errors are detected directly, unlike dynamic techniques where only the presence of an error is detected.

2.1) Code Reading

Code reading is a technique that concentrates on how to read and understand a computer program. It is essential for a software developer to know code reading. The process of reading a software program in order to understand it is known as **code reading or program reading.** In this process, attempts are made to understand the documents, software specifications, or software designs. The purpose of reading programs is to determine the correctness and consistency of the code. In addition, code reading is performed to enhance the software code without entirely changing the program or with minimal disruption in the current functionality of the program. Code reading also aims at inspecting the code and removing (fixing) errors from it.

The general conventions that can be followed while reading the software code are listed below.

- **Figure out what is important:** While reading the code, emphasis should be on finding graphical techniques (bold, italics) or positions (beginning or end of the section). Important comments may be highlighted in the introduction or at the end of the software code. The level of details should be according to the requirements of the software code.
- **Read what is important:** Code reading should be done with the intent to check syntax and structure such as brackets, nested loops, and functions rather than the non-essentials such as name of the software developer who has written the software code.

2.2)Static Analysis

Static analysis comprises a set of methods used to analyze the source code or object code of the software to understand how the software functions and to set up criteria to check its correctness. Static analysis studies the source code without executing it and gives information about the structure of model used, data and control flows, syntactical accuracy, and much more. **Due to this, there are several kinds of static analysis methods, which are listed below.**

- **Data flow analysis:** these are technically not errors and they go undetected by the compiler. These are the symptoms of error caused due to carelessness in typing or error in coding. Ex:-x=a;

x does not appear in right hand side

x=b;

- **Control flow analysis:** This examines the control structures (sequence, selection, and repetition) used in the code. It identifies incorrect and inefficient constructs and also reports unreachable code, that is, the code to which the control never reaches.
- **Data analysis:** This ensures that-proper operations are applied to data objects (for example, data structures and linked lists). In addition, this method also ensures that the defined data is properly used. Data analysis comprises two methods, namely, data dependency and data-flow analysis. **Data dependency** (which determines the dependency of one variable on another) is essential for assessing the accuracy of synchronization across multiple processors. **Dataflow analysis** checks the definition and references of variables.

Other uses of static analysis

Some of the other form of static analysis are,

- An error often made, especially when different teams are developing different parts of software, is mismatched parameter lists, where the argument list of a module invocations different in number or type from the parameters of the invoked module. this can be detected by the compiler if no separate compilation are allowed and the entire program text is available to the compiler.however if programs are separated developed and compiled, which often occurs in large software developments, this error will not be detected. It can be detected only by code review.

2.3) Symbolic Execution

Symbolic execution concentrates on assessing the accuracy of the model by using symbolic values instead of actual data values for input. Symbolic execution, also known as **symbolic evaluation,** is performed by providing symbolic inputs, which produce expressions for the output.

Symbolic execution uses a standard mathematical technique for representing the arbitrary program inputs (variables) in the form of symbols. To perform the calculation, a machine is employed to perform algebraic manipulation on the symbolic expressions. These expressions include symbolic data meant for execution. The symbolic execution is represented as a symbolic state symbol consisting of variable symbolic values, path, and the path conditions. The symbolic state for each step in the arbitrary input is updated. The steps that are commonly followed for updating the symbolic state considering all possible paths are listed below.

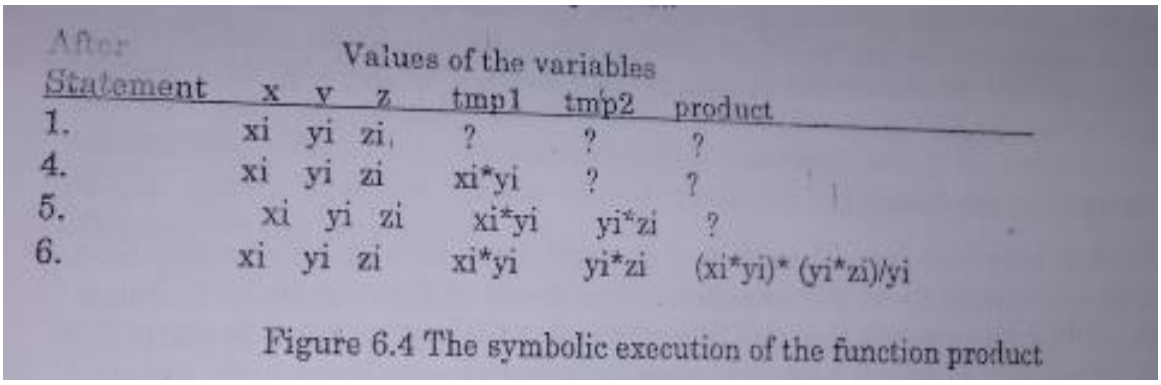
1. The read or the input symbol is created.
2. The assignment creates a symbolic value expression.
3. The conditions in symbolic state add constraints to the path condition.

- A simple program to compute the product of three positive numbers. Let us consider that the symbolic inputs to the function are xi,yi and zi. The aim is to determine the symbolic values of different variables in the program after executing each statement , so that eventually we can determine the result of executing this function.

Example

1. **Function product(x,y,z:integer):integer;**
2. **Var tmp1,tmp2:integer;**
3. **Begin**
4. **tmp1:=x*y;**
5. **tmp2:=y*z;**
6. **Product:=tmp1*tmp2/y**
7. **End**

Figure6.3 : function to determine product.



After Statement	Values of the variables x	y	z	tmp1	tmp2	product
1.	xi	yi	zi	?	?	?
4.	xi	yi	zi	xi*yi	?	?
5.	xi	yi	zi	xi*yi	yi*zi	?
6.	xi	yi	zi	xi*yi	yi*zi	(xi*yi)* (yi*zi)/yi

Figure 6.4 The symbolic execution of the function product

PATH CONDITION

- In symbolic execution, when dealing with conditional execution it is not sufficient to look at the state of the variables of the programs at different statements. As a statement will only be executed if the inputs satisfy certain conditions in which the execution of the program will follow a path that includes the statement.
- To capture this concept in symbolic execution, we require a notion of "path condition"
- Path conditions at a statement gives the conditions the inputs must satisfy for an execution to follow the path so that the statement will be executed.
- Path condition is a Boolean expression over the symbolic inputs that never contain any program variables.It will be represented in a symbolic execution by 'pc'.
- Each symbolic execution begins with pc initialized to true.

Example: if we want to represent :**If C then S1 else S2**

There are two possible paths if C is true then S1 else S2.

For the first case we set the path condition pc to

pc←pc^C

This is the path condition for S1.

For the second case we set the path condition to

pc←pc^~C

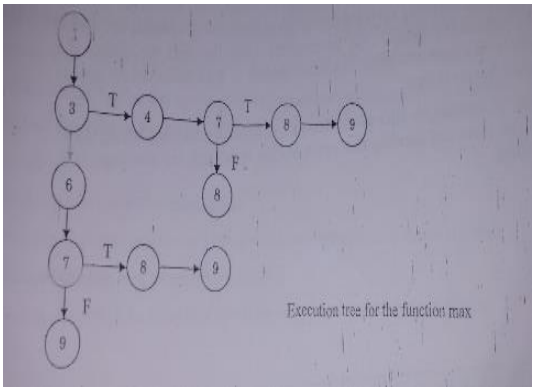
This is the path condition for S2.

Loops and Symbolic Execution trees

- The different paths followed during symbolic execution can be represented by an execution tree.
- A node in this tree represents the execution of a statement
- While arc represents transition from one statement to another.
- For each if statement where both the paths are followed, there are two arcs from the node corresponding to the if statement, one labeled with T(true) another with F(false).

```
1.  function max (x, y, z: integer) : integer
2.  begin
3.    if (x<=y) then
4.      max:=y;
5.    else
6.      max:=x
7.    if max <z then
8.      max:=z;
9.  end;
```

Figure 6.5;The code for function max



Execution tree for the function max

2.4)Proving Correctness

Many techniques for verification aim to reveal errors in the programs, because the ultimate goal is to make programs correct by removing the errors. In proof of correctness, the aim is to prove a program correct. So, correctness is directly established, unlike the other techniques in which correctness is never really established but is implied by the absence of detection of any errors. Proving while developing a program may result in more reliable programs that can be proved more easily.

Code Inspection and Reviews

This technique is a formal and systematic examination of the source code to detect errors. During this process, the software is presented to the project managers and the users for a comment

of approval. Before providing any comment, the inspection team checks the source code for errors. Generally, this team consists of the following.

1. **Moderator:** Conducts inspection meetings, checks errors, and ensures that the inspection process is followed.
2. **Reader:** Paraphrases the operation of the software code.
3. **Recorder:** Keeps record of each error in the software code. This frees the task of other team members to think deeply about the software code.
4. **Author:** Observes the code inspection process silently and helps only when explicitly required. The role of the author is to understand the errors found in the software code.

As mentioned above, the reader paraphrases the meaning of small sections of code during the code inspection process. In other words, the reader translates the sections of code from a computer language to a commonly spoken language (such as English). The inspection process is carried out to check whether the implementation of the software code is done according to the user requirements. **Generally, to conduct code inspections the following steps are performed.**

1. **Planning:** After the code is compiled and there are no more errors and warning messages in the software code, the author submits the findings to the moderator who is responsible for forming the inspection team. The moderator plans the inspection meetings and coordinates with the team members.
2. **Overview:** This is an optional step and is required only when the inspection team members are not aware of the functioning of the project. To familiarize the team members, the author provides details to make them understand the code.
3. **Preparation:** Each inspection team member individually examines the code and its related materials. They use a checklist to ensure that each problem area is checked. Each inspection team member keeps a copy of this checklist, in which all the problematic areas are mentioned.
4. **Inspection meeting:** This is carried out with all team members to review the software code. The moderator discusses the code under review with the inspection team members.

There are two checklists for recording the result of the code inspection, namely, code inspection checklist and inspection error list. The code inspection checklist contains a summary of all the errors of different types found in the software code. This checklist is used to understand the effectiveness of inspection process. **The inspection errorlist** provides the details of each error that requires rework.

All errors in the checklist are classified as major or minor. An error is said to be major if it results in problems and later comes to the knowledge of the user. On the other hand, minor errors are spelling errors and non-compliance with standards.

At the conclusion of the inspection meeting, it is decided whether the code should be accepted in the current form or sent back for rework. In case the software code needs reworking, the author makes all the suggested corrections and then compiles the code. When the code becomes error-free, it is sent back to the moderator. The moderator checks the code that has been reworked. If the moderator is completely satisfied with the software code, inspection becomes formally complete and the process of testing the software code begins.

Unit Testing

Unit testing is a dynamic method for verification, where the program is actually compiled and executed. Unit testing involves executing the code with some test cases and then evaluating the results.

The goal of unit testing is to test modules or “units”, not the entire software system. The programmer, after finishing the coding of a module, tests it with some test data. The tested module is then delivered for system integration and further testing.